

Project 3

zkVM-guest API Standardization

Accepted Standards

1 RISC-V Target Triple *riscv64im_zicclsm-unknown-none-elf*

Defines the minimal 64-bit RISC-V ISA for proving Ethereum blocks. Uses RV64IM for 64-bit arithmetic efficiency and mandates Zicclsm for transparent misaligned memory access handling.

3 Cryptographic Accelerators C Interface *zkvm_accelerators.h*

A portable C API for accessing zkVM-optimized cryptographic operations based on EVM precompiles, abstracting vendor-specific implementations.

5 Memory Layout Restrictions *linker scripts*

Vendors define their own memory layout via linker scripts. Source-code portability is preserved; binary portability is not required.

2 IO Interface *read_input() / write_output()*

Standardizes how guest programs can efficiently read private inputs and write public outputs in a zkVM-independent, zero-copy manner.

4 Execution Termination Semantics *termination*

Specifies what happens when a guest program ends. Successful executions may produce a valid proof; failed executions must not pass verification.

Reminder: Implementation Requirements

zkVM vendors are expected to comply with accepted standards.

64

Migrate to 64-bit RISC-V

The standardized target is RV64IM. Vendors currently running 32-bit zkVMs are expected to plan and execute migration. This is required for compatibility with the standardized target triple.

Ø

Ship a no_std Rust SDK

The target riscv64im-unknown-none-elf is bare-metal with no OS or standard library. Vendor Rust SDKs must be no_std compatible. Any use of std will fail to compile against this target.

IO

Implement standard IO and accelerator interface

Guest programs must be able to call read_input()/write_output() and access the accelerator API as specified. Progress has already been made on accelerator API conformance — we'd love to see the rest of you reach that milestone as well.

The Ethereum Foundation can provide engineering support to vendors that need help with standard implementation — reach out if you do.

New Proposal: Minimum Memory Resources

Needs review and discussion — benchmark values are still to be determined.

Motivation

The Memory Layout Restrictions standard grants vendors ownership of their linker scripts. However, without a floor on available memory, guest authors have no stable target to write against. This standard establishes minimum stack and heap sizes so a guest program that fits within those bounds will run on any conforming zkVM.

Stack

X MB minimum

Every conforming zkVM must provide at least X MB of stack space. The exact value will be determined by profiling the most demanding guest programs under realistic mainnet conditions.

Heap

Y MB minimum

Every conforming zkVM must provide at least Y MB of heap space. Vendors may offer more; guest programs relying on more than the minimum are not guaranteed to be portable.

Vendors must reflect these minimum sizes in their linker scripts.

New Proposal: Memory Safety Guard Regions

Part of a broader hardening initiative. Feedback and review welcome.

Two mandatory guard regions — reserved address ranges where any access triggers abnormal termination, consistent with the Execution Termination Semantics standard.

Null Pointer Trap Region

0x0000 - 0x0FFF (first 4 kB)

Reserved as a trap region. Any read or write access triggers abnormal termination. Catches null pointer dereferences for C, C++, Rust, and Go targets, where null is the zero address.

Stack Guard Region

≥ 4 kB below stack bottom

A contiguous region placed immediately below the bottom of the stack. Catches stack overflows from deep recursion or large stack-allocated objects.

Limitation — Stack Clash: A frame large enough to skip the 4 kB guard in one allocation won't be caught. The compiler flag - **fstack-clash-protection** addresses this but cannot be mandated by a zkVM standard. Guest authors should enable it when available.